IAI600 – Introduction to Applied Artificial Intelligence

Lab III: Classification

Main topic: Classification and Classification Evaluation
Dataset: MNIST Dataset
Software: Python, Jupyter Notebook, NumPy, Matplotlib, Scikit-learn
Disclaimer: This lab material has been prepared with the assistance of Generative AI and is based on Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow; please report any errors, inaccuracies, or limitations you identify.

Objective of the laboratory: The objective of this lab is to introduce classification as a supervised machine learning task. Students will build binary and multiclass classification models, generate predictions, and evaluate their performance using accuracy, cross-validation, confusion matrices, precision, recall, F1-score, and ROC curves.
The laboratory also introduces the important concept that the performance of a classification model depends on the evaluation metric and the decision threshold used to make predictions.

Introduction to Classification

In the previous labs (I and II), we worked with a regression problem in which the model predicted a numerical value. Classification is different.
In a classification problem, the model predicts a **class or category**.
Examples include: spam or not spam, defective or non-defective, fraudulent or legitimate, cat or dog, handwritten digit from 0 to 9.
In this laboratory, we will use the MNIST dataset, which contains images of handwritten digits. Each image belongs to one of ten classes: $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$
The objective is to train a machine learning model that can identify the digit represented by an image.

1 Part I – Load and Explore the MNIST Dataset

Scikit-learn provides the MNIST dataset through the `fetch_openml()` function.
Load the dataset:

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml(
    "mnist_784",
    as_frame=False
)

X, y = mnist["data"], mnist["target"]
```

Inspect the shape of the data:

```
print(X.shape)
print(y.shape)
```

The MNIST images contain 28×28 pixels. Therefore, each image contains: $28 \times 28 = 784$ features.
Display one image:

```
import matplotlib.pyplot as plt

some_digit = X[0]

some_digit_image = some_digit.reshape(
    28, 28
)

plt.imshow(
    some_digit_image,
    cmap="binary"
)

plt.axis("off")
plt.show()
```

Inspect its label:

```
print(y[0])
```

Convert the labels to integers:

```
y = y.astype(np.uint8)
```

Task 1.1 – Explore the Dataset

Answer:

1. How many images are contained in the dataset?
2. How many features does each image contain?
3. How many classes are there?
4. What does each feature represent?
5. What is the target variable?
6. Explain why visual inspection is useful before training a classification model.

Part II – Create Training and Test Sets

The MNIST dataset is already ordered in a way that allows us to use the first part for training and the remaining part for testing.
Use:

```
X_train = X[:60000]
X_test = X[60000:]

y_train = y[:60000]
y_test = y[60000:]
```

For this lab, we will initially focus on distinguishing one digit from all other digits. For example, choose digit 5 as the positive class.
Create binary target labels:

```
y_train_5 = (y_train == 5)
y_test_5 = (y_test == 5)
```

The classification problem is now: **Is this image a 5?**
The possible answers are:

- `True`: the image is a 5,
- `False`: the image is not a 5.

Task 2.1 – Understand Binary Classification

Answer:

1. What is the positive class?
2. What is the negative class?
3. Why is this called binary classification?

Part III – Train a Binary Classifier

We will use Scikit-learn's `SGDClassifier`. Stochastic Gradient Descent (SGD) can be used to train a linear classifier efficiently on large datasets.
Create and train the classifier:

```
from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(
    random_state=42
)

sgd_clf.fit(
    X_train,
    y_train_5
)
```

Test the classifier on an individual image:

```
sgd_clf.predict(
    [some_digit]
)
```

Task 3.1 – Generate Predictions

Test the classifier using several images. For each image, compare: the actual label, whether it is a 5, the model prediction.
Answer:

1. Does the classifier correctly identify all 5s?
2. Does it ever classify a non-5 as a 5?
3. Why is checking only a few predictions not enough to evaluate the model?

Part IV – Accuracy and Cross-Validation

A simple performance measure for classification is **accuracy**. Accuracy is the proportion of predictions that are correct:

$$Accuracy = \frac{\text{number of correct predictions}}{\text{total number of predictions}}$$

Use cross-validation to estimate the classifier's performance.

```
from sklearn.model_selection import cross_val_score

cross_val_score(
    sgd_clf,
    X_train,
    y_train_5,
    cv=3,
    scoring="accuracy"
)
```

Task 4.1 – Evaluate Accuracy

Calculate the mean cross-validation accuracy. Answer:

1. What accuracy did you obtain?
2. Is the accuracy high?
3. Does a high accuracy necessarily mean that the classifier is useful?
4. What happens if the positive class is relatively rare?

Consider the following example: Suppose only 5% of the images are 5s. A classifier that always predicts "not 5" could achieve approximately 95% accuracy while completely failing to identify any 5s. This demonstrates why accuracy alone can be misleading.

Part V – Confusion Matrix

A confusion matrix provides more detailed information about classification performance.
First generate predictions using cross-validation:

```
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(
    sgd_clf,
    X_train,
    y_train_5,
    cv=3
)
```

Calculate the confusion matrix:

```
from sklearn.metrics import confusion_matrix

confusion_matrix(
    y_train_5,
    y_train_pred
)
```

The confusion matrix contains four important quantities: True Positive (TP), True Negative (TN), False Positive (FP), False Negative (FN)
They can be represented as:

	Predicted Positive	**Predicted Negative**
Actual Positive	TP	FN
Actual Negative	FP	TN

Task 5.1 – Interpret the Confusion Matrix

From your confusion matrix, identify: True positives, True negatives, False positives, False negatives. Explain what each of these cases means for the MNIST "5" classification problem.

Part VI – Precision, Recall and F1-Score

Accuracy does not tell the complete story. Two important classification metrics are **precision** and **recall**.
Precision measures the proportion of predicted positives that are actually positive:

$$Precision = \frac{TP}{TP + FP}$$

Recall measures the proportion of actual positives that were correctly identified:

$$Recall = \frac{TP}{TP + FN}$$

Calculate both metrics:

```
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score

precision = precision_score(
    y_train_5,
    y_train_pred
)

recall = recall_score(
    y_train_5,
    y_train_pred
)

print("Precision:", precision)
print("Recall:", recall)
```

The F1-score combines precision and recall:

$$F_1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Calculate it:

```
from sklearn.metrics import f1_score

f1 = f1_score(
    y_train_5,
    y_train_pred
)

print("F1-score:", f1)
```

Task 6.1 – Interpret Precision and Recall

Answer:

1. What does precision tell you?
2. What does recall tell you?
3. What does the F1-score tell you?
4. Which metric would be more important if false positives were very costly?
5. Which metric would be more important if false negatives were very costly?

Give one real-world example where:

- high precision is particularly important;
- high recall is particularly important.

Part VII – Precision/Recall Trade-off

A classification model often produces a score indicating how strongly an example belongs to the positive class. The final prediction depends on a **decision threshold**. Changing the threshold changes the balance between precision and recall.
Obtain decision scores for the training examples:

```
y_scores = cross_val_predict(
    sgd_clf,
    X_train,
    y_train_5,
    cv=3,
    method="decision_function"
)
```

Use different thresholds to generate predictions. For example:

```
threshold = 0

y_some_digit_pred = (
    y_scores >= threshold
)
```

Try different threshold values. Calculate precision and recall for each threshold.

Task 7.1 – Investigate the Threshold

Experiment with several threshold values.
Create a small table:

Threshold	Precision	Recall

Answer:

1. What happens to precision when the threshold increases?

2. What happens to recall?
3. Why does changing the threshold affect the two metrics?
4. Is there one threshold that is always best?

Part VIII – Precision-Recall Curve

The relationship between precision and recall can be visualized using a precision-recall curve. Calculate precision and recall for different thresholds:

```
from sklearn.metrics import precision_recall_curve

precisions, recalls, thresholds = (
    precision_recall_curve(
        y_train_5,
        y_scores
    )
)
```

Plot the relationship:

```
plt.plot(
    thresholds,
    precisions[:-1],
    label="Precision"
)

plt.plot(
    thresholds,
    recalls[:-1],
    label="Recall"
)

plt.xlabel("Threshold")
plt.grid()
plt.legend()
plt.show()
```

Task 8.1 – Interpret the Precision-Recall Curve

Explain:

1. What happens to precision as the threshold increases?
2. What happens to recall?
3. How could a company choose an appropriate threshold?
4. Why might the default threshold not always be appropriate?

Part IX – ROC Curve and ROC-AUC

Another common evaluation tool is the **Receiver Operating Characteristic (ROC) curve**. The ROC curve plots: True Positive Rate (TPR) (also called recall), and False Positive Rate (FPR).
Calculate the ROC curve:

```
from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(
    y_train_5,
    y_scores
)
```

Plot the curve:

```
plt.plot(
    fpr,
    tpr,
    linewidth=2
)

plt.plot(
    [0, 1],
    [0, 1],
    "k--"
)

plt.xlabel("False␣Positive␣Rate")
plt.ylabel("True␣Positive␣Rate")
plt.grid()
plt.show()
```

Calculate the Area Under the ROC Curve:

```
from sklearn.metrics import roc_auc_score

roc_auc = roc_auc_score(
    y_train_5,
    y_scores
)

print("ROC-AUC:", roc_auc)
```

Task 9.1 – Interpret ROC-AUC

Answer:

1. What does the ROC curve represent?
2. What do the x-axis and y-axis represent?
3. What does a ROC-AUC value close to 1 indicate?
4. What does a value close to 0.5 indicate?

Part X – Multiclass Classification

So far, we have performed binary classification: 5 vs. not 5
MNIST naturally contains ten classes. This is a **multiclass classification** problem.
Train the SGD classifier using the original labels:

```
sgd_clf.fit(
    X_train,
    y_train
)
```

Predict an image:

```
sgd_clf.predict(
    [some_digit]
)
```

Inspect the decision scores:

```
some_digit_scores = (
    sgd_clf.decision_function(
        [some_digit]
    )
)

print(some_digit_scores)
```

Identify the predicted class:

```
print(
    sgd_clf.classes_
)
```

Task 10.1 – Understand Multiclass Classification

Answer:

1. How many classes are being predicted?
2. What does the decision score represent?
3. Which class has the highest score?
4. Does the model need to be trained separately for every class?

Briefly investigate how Scikit-learn handles multiclass classification for `SGDClassifier`.

Part XI – Evaluate the Multiclass Classifier

Generate predictions using cross-validation:

```
y_train_pred = cross_val_predict(
    sgd_clf,
    X_train,
    y_train,
    cv=3
)
```

Calculate the confusion matrix:

```
conf_mx = confusion_matrix(
    y_train,
    y_train_pred
)

print(conf_mx)
```

Visualize the confusion matrix:

```
plt.matshow(
    conf_mx,
    cmap=plt.cm.gray
)

plt.show()
```

Task 11.1 – Analyze Classification Errors

Inspect the confusion matrix. Answer:

1. Which digits appear to be confused most often?
2. Which digits appear to be easier to classify?
3. Why can a confusion matrix be more informative than a single accuracy value?

Part XII – Multilabel Classification

In some classification problems, an observation can belong to more than one class at the same time. This is called **multilabel classification**.
For example, an image could be labeled according to whether it:

- contains an odd digit;
- contains a digit greater than 5.

Create two labels:

```
y_train_large = (
    y_train >= 7
)

y_train_odd = (
    y_train % 2 == 1
)

y_multilabel = np.c_[
    y_train_large,
    y_train_odd
]
```

Train a classifier:

```
from sklearn.neighbors import KNeighborsClassifier

knn_clf = KNeighborsClassifier()

knn_clf.fit(
    X_train,
    y_multilabel
)
```

Make a prediction:

```
knn_clf.predict(
    [some_digit]
)
```

Task 12.1 – Understand Multilabel Classification

Answer:

1. What makes this problem different from ordinary multiclass classification?
2. How many labels can an observation receive?
3. What does the prediction for a single image represent?

Part XIII – Multilabel Evaluation

Evaluate the multilabel classifier using an appropriate metric.
For example:

```
y_train_knn_pred = cross_val_predict(
    knn_clf,
    X_train,
    y_multilabel,
    cv=3
)
```

Calculate the F1-score:

```
f1_score(
    y_multilabel,
    y_train_knn_pred,
    average="macro"
)
```

Task 13.1 – Interpret the Result

Discuss:

1. Why is evaluating a multilabel classifier different from evaluating a binary classifier?
2. Why should the evaluation metric be selected according to the problem?